

RESEARCH ARTICLE

Toward Efficient Deobfuscation via Large Language Models

BYUNGGEON CHOI^{1,2}, HONGJOO JIN³, DONG HOON LEE¹, AND WONSUK CHOI¹¹School of Cybersecurity, Korea University, Seongbuk-gu, Seoul 02841, Republic of Korea²Samsung SDS, Seoul 05510, Republic of Korea³Samsung Electronics, Suwon-si, Gyeonggi-do 16677, Republic of Korea

Corresponding author: Wonsuk Choi (beb0396@korea.ac.kr)

This work was partly supported by the Institute of Information and Communications Technology Planning and Evaluation (IITP) grant funded by the Ministry of Science and ICT (MIST) (RS-2024-00399389, Generative AI based Binary Deobfuscation Technology and Its Evaluation Metrics, 50%) and the Institute of Information and Communications Technology Planning and Evaluation (IITP) grant funded by the Ministry of Science and ICT (MIST) (2022-0-00979, Development of technology and test criteria for evaluating the security of self-driving vehicle data and V2X communication network, 50%).

ABSTRACT Obfuscation modifies code structure to impede reverse engineering and is widely used to protect intellectual property and evade malware detection. However, existing deobfuscation techniques largely rely on analyst-driven heuristics or are tailored to specific tools, suffering from low adaptability, scalability, and automation when handling complex or nested obfuscation methods. The core objective of this study is to develop an automated and scalable deobfuscation methodology that can generalize across diverse and complex nested obfuscation techniques by leveraging the capabilities of Large Language Models (LLMs). To achieve this, we present LLM4DOBF, a novel methodology that combines LLM fine-tuning to learn various obfuscation patterns with optimized prompt engineering. This approach guides the LLM to effectively analyze obfuscated code and restore it to its original form. Experimental results demonstrate that LLM4DOBF achieves superior code deobfuscation quality, as validated by high SacreBLEU scores, and shows notable code optimization improvements in the ‘Obfuscation Quality Quantification Framework’ evaluation. Furthermore, it consistently outperformed other LLMs in complex scenarios using major obfuscation tools like Tigress and OLLVM, proving its stability with a high response rate and a low compilation error rate. This study confirms that the proposed LLM4DOBF overcomes the limitations of existing heuristic-based methods, providing a robust, reliable, and generalizable solution for automated deobfuscation. This constitutes a significant contribution by presenting a scalable approach for modern obfuscated code analysis.

INDEX TERMS Source code obfuscation, source code deobfuscation, large language models, prompt engineering, malware.

I. INTRODUCTION

Obfuscation modifies a program’s internal structure while preserving its original functionality, thereby making reverse engineering considerably more difficult [1], [2]. Although software developers utilize obfuscation to protect intellectual property, malicious actors frequently exploit these same techniques to evade detection [3], [4]. Studies indicate that 99% of malicious code employs obfuscation, complicating

malware analysis and escalating the computational overhead required for detection [5], [6], [7]. As obfuscation methods continue to evolve, the prevalence of obfuscated malware increases, further amplifying analytical complexity [8], [9].

In response to these challenges, researchers have proposed various deobfuscation [10], [11], [12], [13], [14], [15] techniques designed to restore obfuscated code to its original form. However, current approaches face two primary limitations. First, they depend heavily on analyst-driven heuristics. Many methods require manual identification of obfuscation patterns [16], [17], [18], especially in complex

The associate editor coordinating the review of this manuscript and approving it for publication was Mostafa M. Fouda¹.

transformations such as Mixed Boolean Arithmetic (MBA) [19], where analysts must pinpoint obfuscated expressions before applying formula rewriting or program synthesis. Second, existing deobfuscation approaches are typically designed with a focus on specific tools [20], [21], [22], which limits their ability to effectively process multiple obfuscation methods applied within a single code. Kochberger et al. [23] showed that specific deobfuscation strategies excel under particular conditions, but their effectiveness declines in other environments, underscoring their limited generalizability. These constraints inhibit the scalability and automation of deobfuscation workflows.

Recent advances in large language models (LLMs) offer a promising alternative. LLMs have demonstrated strong performance in natural language processing and software engineering tasks, including code analysis [24], debugging [25], and fuzzing [26], [27]. However, directly applying an LLM to deobfuscation may yield inconsistencies or challenges [28] in reconstructing complex code structures. To realize their full potential for deobfuscation, rigorous fine-tuning on diverse obfuscation patterns and meticulous prompt engineering are essential [29], [30], [31]. By training on datasets containing obfuscated and corresponding original code, an LLM can learn generalized representations of varying obfuscation techniques. At the same time, optimized prompts enhance the model's understanding of obfuscation schemes, guiding it to produce accurate deobfuscated code.

This study presents LLM4DOBF,¹ a novel methodology that leverages LLMs for effective code deobfuscation. Our approach follows a structured process to bolster deobfuscation performance. First, we fine-tune a pre-trained LLM (e.g., GPT-3.5-turbo) on a dataset comprising obfuscated and original code samples. This enables it to recognize patterns in MBA, Control Flow Flattening, Opaque Predicates, and other areas. Next, we employ optimized prompt engineering with explicit Contextual Statements tailored to each obfuscation technique, ensuring a more precise understanding of the corresponding structural transformations. Finally, we evaluate performance using SacreBLEU scores and the Obfuscation Quality Quantification Framework, measuring structural similarity and optimization levels. This systematic pipeline allows LLM4DOBF to support a broad range of obfuscation strategies while mitigating the shortcomings of traditional methods.

The proposed methodology offers several advantages. Unlike heuristic-driven approaches, LLM4DOBF operates autonomously and significantly reduces the need for human intervention. The model also supports continual learning, enabling it to adapt to emerging obfuscation methods by integrating new obfuscation patterns into its training corpus. Moreover, its general-purpose framework can effectively handle multiple tools and combinations of obfuscation methods.

The key contributions of this paper are as follows:

- **Universal Deobfuscation Method:** LLM4DOBF combines fine-tuning and prompt engineering to yield a flexible, scalable deobfuscation model that transcends the limitations of methods designed for specific obfuscation types.
- **Automation and Generalization:** By minimizing reliance on manual analysis, LLM4DOBF provides a fully automated solution capable of handling diverse obfuscation strategies and varying implementation details by reducing dependence on manual analysis.
- **Comprehensive Performance Evaluation:** Rigorous experimentation using SacreBLEU scores and the Obfuscation Quality Quantification Framework confirms the model's effectiveness and reliability across multiple obfuscation environments.

II. BACKGROUND

A. SOURCE CODE OBFUSCATION

Obfuscation is designed to intentionally reduce the readability and ease of analyzing software code. It makes reverse engineering more challenging and enhances security. Source code obfuscation is primarily used to protect software intellectual property, conceal sensitive algorithms, and prevent malicious users or attackers from analyzing internal logic and exploiting vulnerabilities. Source code obfuscation methods apply algorithmic and structural transformations to obscure the original structure and intent of the code. In particular, modern obfuscation methods go beyond simple modifications such as renaming variables or removing comments [33]; they fundamentally reconfigure the logical structure of the code, making execution flows and calculation processes more complex. During the obfuscation process, the code retains its original functionality, but its internal representation becomes significantly more intricate, posing challenges for both human understanding and automated analysis. As obfuscation methodology evolves, deobfuscation methods aimed at bypassing or decoding it also continue to advance.

Even when using the same obfuscation methods, differences in implementation across obfuscation tools make standardized countermeasures difficult [34], [35], [36]. Each tool applies unique algorithms and transformation strategies, leading to variations in code structure and obfuscation patterns [20], [37]. As a result, even when identical obfuscation methods are applied, the resulting obfuscated code may differ depending on the tool, introducing further complexity to the deobfuscation process.

Obfuscation methods can be implemented through various methods. This study focuses on three representative source code obfuscation methods—Mixed Boolean-Arithmetic [19], Control Flow Flattening [38], and Opaque Predicates [39].

1) MIXED BOOLEAN-ARITHMETIC

Mixed Boolean-Arithmetic (MBA) is an obfuscation method that combines arithmetic and logical operations to increase

¹This paper is a revised and extended version of the LLM4DOBF [32] paper presented at WISA 2024.

the complexity of a program's computational processes. The core idea behind this method is to obscure the explicit meaning of the code by transforming simple operations into multiple layers of mixed expressions. For example, a basic addition operation can be replaced with a more intricate expression that interweaves arithmetic operations (e.g., addition, subtraction) and Boolean operations (e.g., AND, OR, XOR). While the transformed expression yields the same result as the original, it becomes significantly more challenging for humans to analyze and interpret. A defining characteristic of MBA is its ability to make computational flows in source code exceedingly complex, posing substantial challenges for reverse engineers attempting to decipher it. To understand the actual behavior of the code, a reverse engineer must either revert the transformed expressions to their original form or manually compute each operation's result. One of the MBA method's strengths lies in its strategic use of mathematical properties to express the same computation in multiple ways, further complicating analysis. When a simple arithmetic operation is rewritten using MBA, its intended function becomes far less intuitive. As a result, analysts must spend considerably more time interpreting the code, and in some cases, they may not fully comprehend its logic.

2) CONTROL FLOW FLATTENING

Control Flow Flattening is an obfuscation method that deliberately distorts a program's execution flow, making its behavior challenging to understand. This method transforms the program's original execution flow—typically a simple sequential structure—into a complex, non-linear form. This prevents straightforward predictions of execution order and intuitive comprehension of program logic. The core principle of this method involves breaking down the execution flow into multiple states and controlling transitions between them using conditional or branching statements. In a conventional program structure, the order in which specific blocks or functions are executed is relatively easy to determine. However, when Control Flow Flattening is applied, this clarity disappears. Instead, all execution blocks are placed on a single “flattened” control flow plane. Execution depends on a state variable and conditional statements, making it extremely difficult to predict the program's flow externally. By restructuring execution flow into a non-linear form, Control Flow Flattening significantly complicates static analysis, maximizes the obfuscation effect, and makes reverse engineering far more challenging.

3) OPAQUE PREDICATES

Opaque Predicates is an obfuscation method that artificially increases the complexity of source code by introducing conditional statements whose outcomes remain fixed, regardless of actual execution. This method inserts unnecessary conditional branches into the program's execution flow, making analysis more difficult for both reverse engineers and automated tools. While Opaque Predicates may appear

to determine execution results dynamically, their evaluation outcome is always constant. By obscuring the actual execution flow, they effectively conceal the original intent of the code. This method is particularly effective at confusing static analysis tools and can be implemented in various ways. A common approach is to design conditional statements using mathematical properties or seemingly nondeterministic random expressions. Although analysis tools attempt to evaluate the logic of these conditions to infer the execution path, the fact that Opaque Predicates always return a fixed result makes them inherently difficult to interpret.

B. LARGE LANGUAGE MODELS (LLMs)

Large Language Models (LLMs) are artificial intelligence models that learn from vast amounts of text data to understand linguistic context, achieving high performance in tasks such as text generation, summarization, translation, and analysis. Representative examples include OpenAI's GPT series and Google's Gemini model, both of which leverage billions to trillions of parameters and are trained on large-scale datasets to learn complex language patterns effectively. LLMs drive innovation in Natural Language Processing (NLP) and have applications across various domains [43]. Beyond conversational AI, content generation, code auto-completion, and data summarization, their potential in security and cybersecurity is rapidly growing. In particular, research on applying LLMs to software security is actively advancing [44], [45], [46].

In the security domain, LLMs are commonly used for malicious code detection, vulnerability analysis, and automated code review [47], [48]. LLMs can identify patterns or anomalies by analyzing code semantics and structure, enabling automated detection. For instance, in malicious code detection, LLMs offer higher accuracy and broader applicability than traditional signature-based detection methods due to their ability to learn and generalize from diverse malicious code patterns [49]. In vulnerability analysis, LLMs assist in identifying potential defects within code and proactively discovering security weaknesses. They excel at scanning extensive code-bases and detecting recurring vulnerabilities.

However, when an existing LLM is directly applied to deobfuscation, it may fail to fully capture code context or produce inconsistent outputs [28]. To ensure reliable deobfuscation, a custom approach that considers the unique attributes of obfuscated code is essential—simply deploying the model is insufficient. In other words, LLMs require precise training and optimization to effectively analyze obfuscation patterns and restore the original logic of the code.

C. PROMPT ENGINEERING FOR LLMs

Maximizing the performance of Large Language Models (LLMs) requires carefully designing the prompts provided by users. This process, known as prompt engineering, has become a key method for ensuring that an LLM accurately interprets user intent and generates the desired results [50],

TABLE 1. Comparative analysis of prominent LLMs (GPT-3.5-turbo-0125, GPT-4o, Gemini-1.5-flash, and Claude-3.5-sonnet).

Model	Release Date	Provider	Parameters	Cost(In/Output token)	Architecture
GPT-3.5-turbo-0125 [40]	2024.01	OpenAI	175 B	\$0.5 / \$1.5	Decoder
GPT-4o [40]	2024.05	OpenAI	1.7 T	\$2.5 / \$10	Decoder
Gemini-1.5-flash [41]	2024.05	Google	137 B	Free	Encoder-Decoder
Claude-3.5-sonnet [42]	2024.06	Anthropic	Unknown	\$3 / \$15	Decoder

[51]. Since LLMs are developed as general-purpose models rather than being predefined for specific tasks, the structure, context, and clarity of the prompt directly impact output quality. For example, a simple prompt such as “Please deobfuscate this code” may produce ambiguous results, whereas a more structured and detailed prompt—like the one described in Section III-C—can yield more reliable outcomes.

Prompt engineering significantly enhances the efficiency of LLMs and plays a vital role in various domains, including security and obfuscation. A well-constructed prompt minimizes unnecessary computation, allowing the model to better understand the context of complex problems and perform tasks more effectively. In particular, for intricate tasks such as interpreting and restoring obfuscated code, prompts help guide the model in recognizing obfuscation methods, reconstructing execution flows, and accurately grasping the structural meaning of code. When domain expertise is seamlessly integrated with prompt design methods, LLMs can deliver more dependable results. The continued development of prompt engineering will serve as a fundamental approach to leveraging LLMs for security analysis and deobfuscation.

III. APPROACH

In this section, we present LLM4DOBF, a novel methodology leveraging large language model (LLM) to overcome the complexities of deobfuscation in source code. This approach provides a scalable and automated deobfuscation solution by reducing reliance on heuristics and effectively adapting to evolving obfuscation techniques.

A. MODEL SELECTION

This study selects OpenAI’s GPT-3.5-turbo-0125 as the large language model (LLM) for implementing deobfuscation. The selection process considers multiple factors, including performance, cost-effectiveness, applicability of the latest methods, and feasibility in real-world settings. As shown in Table 1, GPT-3.5-turbo-0125 comprises 17.5 billion parameters, which is sufficient to effectively analyze and process complex obfuscation methods. Additionally, it provides a cost-efficient solution for large-scale applications, with pricing set at \$0.5 per 1M input tokens and \$1.5 per 1M output tokens.

In comparison, GPT-4o demonstrates superior performance with 1.7 trillion parameters but incurs a higher token cost ranging from \$2.5 to \$10. Anthropic’s Claude-3.5 Sonnet faces similar cost constraints, with its parameter count remaining undisclosed. Meanwhile, Google’s

Gemini-1.5-flash is available free of charge, offering an advantage in terms of cost. However, it has only 13.7 billion parameters and exhibits limited stability, which may reduce performance in precise deobfuscation tasks.

Released in January 2024, GPT-3.5-turbo-0125 is a recent model that offers enhanced language processing capabilities and has been widely adopted in research. Its decoder-based architecture is particularly suited for simplifying the structural complexity of obfuscated code and reconstructing execution flows. Furthermore, its efficient token generation enables stable output even in repetitive tasks, preventing performance degradation.

Consequently, GPT-3.5-turbo-0125 is selected as the optimal model for the LLM4DOBF method in this study due to its balanced performance and cost, adaptability to the latest methods, and structural advantages for deobfuscation. This choice enhances the practicality and reproducibility of the research while maximizing both efficiency and accuracy in deobfuscation.

B. FINE TUNING

We fine-tune the GPT-3.5-turbo-0125 model to maximize the performance of LLM4DOBF. According to OpenAI’s official documentation [52], a minimum of 10 datasets is required for fine-tuning, while 50 to 100 datasets are recommended for achieving more significant performance improvements. Following these guidelines, we optimize the model using a dataset consisting of obfuscated code and their corresponding original code pairs.

Specifically, for each obfuscation method, we prepare ten obfuscated code–original code pairs, collected from two benchmark datasets. This process yields twenty code pairs per method. This study considers seven obfuscation methods, including Mixed Boolean-Arithmetic, Control Flow Flattening, and Opaque Predicate, each applied using Tigress and Obfuscator LLVM. Consequently, a total of 280 code pairs (10 pairs $\times$ 2 datasets $\times$ 7 obfuscation methods $\times$ 2 obfuscation tools) are used for fine-tuning.

The primary objective of fine-tuning is to enable the model to learn obfuscation patterns, allowing it to more effectively restore the original code. We maximize the diversity of obfuscation methods and construct a sufficiently large dataset, ensuring that the model generalizes well across various obfuscation techniques. Additionally, we diversify the combinations of obfuscation tools and methods to prevent bias toward specific patterns and improve adaptability to diverse obfuscation strategies.

TABLE 2. Prompt engineering patterns and examples.

Prompt Pattern	Example Prompt
Meta Language Creation	The following obfuscated code has been subjected to Mixed Boolean-Arithmetic (MBA) obfuscation. MBA obfuscation combines arithmetic and Boolean operations. For example, $a+b = (a\oplus b)+2\cdot(a\wedge b)$. Now, analyze the obfuscated code and deobfuscate it to restore its original meaning.
Persona	Now, you are a security expert. Analyze the obfuscated code from the perspective of a security expert, deobfuscate it to restore its original meaning, and evaluate the functionality of the deobfuscated code.
Question Refinement	If parts of the obfuscated code are unclear, generate additional questions to gather more information. For example, generate and answer questions such as, "What is the original variable name in this part?"
Alternative Approaches	There can be multiple approaches to deobfuscating and restoring the obfuscated code. Compare the following two approaches that explain the pros and cons of each: 1. Deobfuscating code through static analysis 2. Deobfuscating code through dynamic analysis Present the results of each approach and ultimately select the most suitable method.

This dataset configuration and fine-tuning strategy are designed to enhance the performance and adaptability of LLM4DOBF, enabling it to effectively handle different obfuscation methods and tools.

C. PROMPT ENGINEERING STRATEGY

Prompt engineering is a core method for maximizing the performance of large language models (LLMs), playing a crucial role in solving complex problems and optimizing model outputs. A study by White et al. [53] proposes and verifies various prompt patterns, demonstrating their effectiveness in improving both response quality and reliability. This study introduces 16 prompt patterns, showing that each can be used independently or in a mutually complementary manner, depending on specific tasks and application domains.

These proposed patterns primarily aim to enhance an LLM's ability to understand context, assume roles, refine questions, and explore alternative solutions. Rather than simply modifying how inputs are structured, they leverage an LLM's stored knowledge and reasoning capabilities, prompting deeper engagement with the problem and generating more refined outputs. In particular, these patterns focus not only on retrieving information but also on producing accurate and practical results in complex tasks such as code analysis and deobfuscation.

In this study, we adopt four of the 16 proposed patterns—*Meta Language Creation Pattern*, *Persona Pattern*, *Question Refinement Pattern*, *Alternative Approaches Pattern*. These patterns are selected based on their alignment with the unique characteristics and requirements of deobfuscation tasks, which involve simplifying code complexity, recognizing patterns, and restoring original code logic. Consequently, these

selected patterns are well-suited for analyzing obfuscated code structures and effectively reconstructing them.

Our prompt engineering strategy extends beyond simply applying these patterns; rather, it aims to maximize their practical impact in real-world applications. By eliminating unnecessary complexity, we optimize the model to deliver more efficient and precise outputs in deobfuscation tasks. While each of the four chosen patterns is individually effective, they are designed to function synergistically throughout the entire deobfuscation process, thereby ensuring high overall performance.

To further enhance the practical applicability of each pattern, this study introduces Contextual Statements to concretely integrate these patterns into deobfuscation tasks. By structuring questions effectively and exploring diverse solutions through Contextual Statements, we contribute to improving LLM4DOBF's performance. Table 2 presents the four adopted prompt patterns along with real-world usage examples, providing a detailed approach to prompt design for maximizing LLM performance in deobfuscation tasks. Additionally, it outlines the specific roles and effects of each selected pattern, offering deeper insights into their contribution to LLM4DOBF's prompt engineering.

1) META LANGUAGE CREATION PATTERN

Meta Language Creation Pattern is designed to introduce new notation or alternative semantics in interactions with the LLM, enabling the effective communication of complex or domain-specific concepts. This pattern allows users to define a "meta language" that the LLM can interpret and apply within the conversation, making it particularly useful for concisely and clearly representing intricate ideas, such as graph structures or instruction sets. In deobfuscation tasks, this pattern is especially effective for clarifying the semantics of complex code transformations, directing the LLM's attention toward the logical structure of the code rather than verbose linguistic descriptions. Consequently, even highly intricate code-related tasks can be handled with greater efficiency and accuracy.

Contextual Statements

The following obfuscated code has been subjected to $\mathbf{X}$ obfuscation. $\mathbf{X}_1$ obfuscation combines $\mathbf{Y}_1$. For example, $\mathbf{Z}_1$. Now, analyze the obfuscated code and deobfuscate it to restore its original meaning.

2) PERSONA PATTERN

Persona Pattern assigns a specific role or perspective to an LLM, guiding it to generate outputs that align with a given context. For example, configuring the LLM as a "security analyst" or "software engineer" encourages the model to focus on domain-specific details, thereby producing contextually rich outputs. In deobfuscation tasks, setting the LLM to act as a "security analyst" directs its attention to security-related considerations, ensuring precise results that are crucial for restoring the original structure. Additionally, this pattern can assign non-human roles, such as "compiler"

or “runtime environment”, allowing the model to more accurately reflect the task’s context and achieve high-level performance in domain-specific applications.

Contextual Statements

Now, you are a X_2 . Analyze the obfuscated code from the perspective of a X_2 , deobfuscate it to restore its original meaning, and evaluate the functionality of the deobfuscated code.

3) QUESTION REFINEMENT PATTERN

Question Refinement Pattern is a method that enables the LLM to enhance or restructure the user’s initial query, ultimately producing more accurate and specific results. This pattern is particularly useful when the user’s question is unclear or unstructured, as it allows the LLM to request additional context or decompose a complex query into simpler sub-questions. In deobfuscation tasks, this approach enables the LLM to prompt the user for further details regarding the obfuscation method and desired output format, thereby facilitating a more efficient and structured workflow. This iterative refinement process reduces ambiguity, enhances response accuracy, and ensures reliable answers even for highly complex challenges.

Contextual Statements

If parts of the obfuscated code are unclear, generate additional questions to gather more information. For example, generate and answer questions such as, “ X_3 ?”

4) ALTERNATIVE APPROACHES PATTERN

Alternative Approaches Pattern is a method that presents multiple problem-solving strategies, allowing the user to select the most suitable approach based on specific requirements. By prompting the LLM to explore solutions beyond those initially considered by the user, this pattern helps mitigate cognitive biases and encourages more innovative problem-solving. In deobfuscation tasks, the LLM can propose various restoration methods (e.g., static analysis, dynamic analysis) and compare their respective advantages and limitations, enabling users to choose the method best aligned with their objectives. This ability to provide alternative perspectives facilitates a comprehensive evaluation of factors such as performance, reliability, and computational overhead, ultimately maximizing the accuracy and efficiency of deobfuscation.

Contextual Statements

There can be multiple approaches to deobfuscating and restoring the obfuscated code. Compare the following two approaches that explain the pros and cons of each:

1. Deobfuscating code through X_4
2. Deobfuscating code through Y_2

Present the results of each approach and ultimately select the most suitable method.

D. LLM4DOBF

LLM4DOBF introduces a novel approach that leverages large language model (LLM) to analyze the structural complexity of obfuscated code and restore its original meaning. Designed for high performance and accessibility,

it ensures reproducible results across various research contexts. To effectively address the complexity and diversity of obfuscation methods, LLM4DOBF follows a systematic process that integrates prompt engineering and fine-tuning. This approach overcomes the limitations of existing deobfuscation methods, which rely primarily on static analysis or rule-based techniques, by utilizing the contextual understanding and reasoning capabilities of LLMs to handle even highly intricate obfuscation methods.

The primary objective of LLM4DOBF is to employ a fine-tuned model that has learned the relationship between obfuscated code and its original counterpart, enabling it to restore the original code from a given obfuscated input. It is designed to process not only individual obfuscation methods, such as Mixed Boolean-Arithmetic, Control Flow Flattening, and Opaque Predicates, but also more complex obfuscation scenarios involving nested obfuscation techniques. To enhance generalization and restoration accuracy, the model is trained on code pairs generated from various obfuscation methods and tools, including Tigress and Obfuscator LLVM.

As illustrated in Figure 1, LLM4DOBF consists of three main processes. The first is prompt engineering, which involves designing input prompts tailored to the characteristics of obfuscated code and the target restoration task. To this end, *Meta Language Creation Pattern*, *Persona Pattern*, *Question Refinement Pattern*, *Alternative Approaches Pattern* are employed in an integrated and complementary manner, maximizing the LLM’s contextual reasoning abilities. The second process is fine-tuning, where the model learns obfuscation-to-original code patterns using an API-based training approach, enabling it to effectively understand and restore even highly complex obfuscation methods. In the final stage, the fine-tuned model restores the input obfuscated code and evaluates the quality of the restored output to generate the final result.

By integrating the generative capabilities of LLM with the structured design of prompt engineering, LLM4DOBF introduces a new method capable of handling complex obfuscation scenarios that existing deobfuscation methods struggle to process. In doing so, LLM4DOBF expands the scope of deobfuscation applications and provides efficient solutions for code security and software analysis.

IV. EXPERIMENTAL SETUP

The experiments in this study were conducted to evaluate the performance of LLM4DOBF, which is based on GPT-3.5-turbo-0125. Our experiments are conducted on a testbed machine equipped with a 12th Gen Intel(R) Core(TM) i7-12700 CPU, 16GB RAM, and an NVIDIA GeForce RTX 3070 Ti GPU. The software environment includes Windows 11 and Python version 3.12.7.

A. DATA COLLECTION

A dataset for experimentation is constructed by generating pairs of obfuscated code and corresponding original code

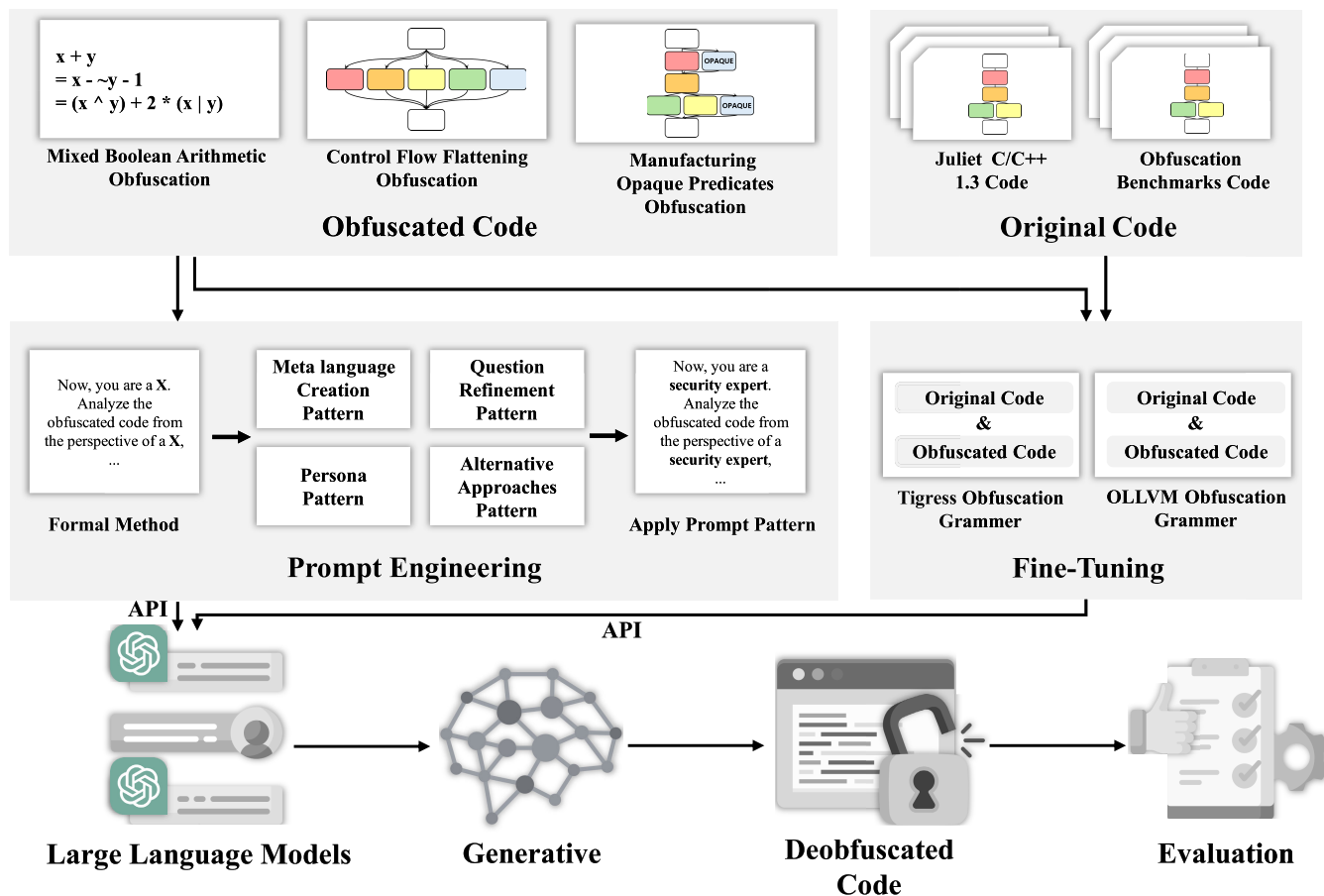


FIGURE 1. Overview of LLM4DOBF.

using various obfuscation methods. The dataset is primarily derived from open-source projects and public datasets, where obfuscation methods are directly applied to the original code to ensure a clear correspondence between the obfuscated and original versions. To achieve this, two obfuscation tools—Tigress and Obfuscator LLVM—are employed. In particular, Obfuscator LLVM operates by obfuscating the intermediate representation (IR) before decompiling it back into C, thereby reconstructing the obfuscated code. This approach effectively captures the diverse characteristics of different obfuscation methods while also simulating realistic obfuscation scenarios.

The dataset is sourced from Juliet C/C++ 1.3 [54] and publicly available Obfuscation Benchmarks on GitHub [55]. Juliet C/C++ 1.3, published by the National Institute of Standards and Technology (NIST), consists of C/C++ examples covering 118 Common Weakness Enumeration (CWE) cases. We select 70 C programs that are compilable in a Linux environment. Additionally, 70 more samples are selected from Obfuscation Benchmarks, comprising 40 basic algorithms and 30 small programs, yielding a total of 140 original code samples.

The obfuscation dataset is organized into seven levels based on three primary obfuscation methods: Mixed Boolean-Arithmetic, Control Flow Flattening, and Manufacturing

Opaque Predicates. Level 1 consists of datasets where each method is applied individually once, Level 2 combines two of these methods, and Level 3 incorporates all three methods simultaneously. Each obfuscation method is applied using both Tigress and Obfuscator LLVM, resulting in seven distinct obfuscated datasets for each of the 140 code samples across both tools. Consequently, the dataset provides a balanced collection of samples covering various combinations of obfuscation methods and tools, establishing a robust foundation for training and evaluation across a wide range of obfuscation scenarios.

B. ESTABLISHING A FAIR COMPARISON

To ensure a fair comparison of deobfuscation performance across different large language models (LLMs), we establish rigorous evaluation criteria. Since LLMs can automatically generate comments or utilize them for interpretive purposes, there is a potential risk of bias when assessing deobfuscation performance. To mitigate this issue and maintain evaluation fairness, all comments are removed from the input code, allowing for a more accurate assessment of LLM4DOBF's code restoration capability.

We obtain valid API keys from each LLM's official platform and exclusively use the official models provided by these services. This approach ensures that non-official

environment variables or non-standard settings do not influence the LLM's performance. Additionally, we construct a Python-based automated testing environment using example code from each platform's official documentation, ensuring a consistent and reproducible workflow for obfuscating code, restoring it, and comparing results.

To maintain fairness and objectivity, all LLMs are evaluated under identical conditions, ensuring that comparisons between LLM4DOBF and other models remain valid and unbiased. This standardized evaluation framework provides a reliable basis for assessing LLM4DOBF's performance across various obfuscation methods and tools.

V. EVALUATION

In this section, we comprehensively evaluate the performance of LLM4DOBF across various dimensions. First, we analyze the impact of different Prompt Engineering strategies on SacreBLEU Scores, highlighting how the inclusion of multiple patterns significantly enhances the model's deobfuscation performance (Section V-A). We analyze SacreBLEU Scores and employ the Obfuscation Quality Quantification Framework to assess the structural and semantic accuracy of deobfuscated code, as well as the code optimization, complexity reduction, and structural simplification achieved through LLM4DOBF (Sections V-B and V-C). Finally, we assess the response reliability and compile error rates of LLM4DOBF in comparison to other LLMs, demonstrating its superior performance in handling diverse and complex obfuscation scenarios (Section V-D).

A. IMPACT OF PROMPT ENGINEERING VARIATIONS

In this experiment, we compare the deobfuscation performance of the GPT-3.5-turbo model with and without the application of Prompt Patterns, using SacreBLEU Scores for evaluation. The comparison is organized into three scenarios: (1) no prompt pattern applied, (2) only *Meta Language Creation Pattern* applied, and (3) all four patterns (*Meta Language Creation Pattern*, *Persona Pattern*, *Question Refinement Pattern*, *Alternative Approaches Pattern*) applied.

As summarized in Figure 2, we measure the average SacreBLEU Scores for obfuscated code generated by Tigress and Obfuscator LLVM, then evaluate the deobfuscation performance. Since *Meta Language Creation Pattern* is considered the most impactful pattern [53], using it alone yields higher SacreBLEU Scores than using no pattern at all. In particular, when all four patterns are applied to code produced by the Tigress obfuscation tool, the model achieves the highest average SacreBLEU Scores, indicating a more pronounced increase in deobfuscation performance.

In contrast, for OLLVM, the difference in performance between applying and not applying the Prompt Patterns is relatively smaller. Even so, using all four patterns produces comparatively high SacreBLEU Scores and maintains stable restoration performance. These results suggest that Prompt Engineering contributes to the model's ability to learn complex code patterns under various obfuscation methods

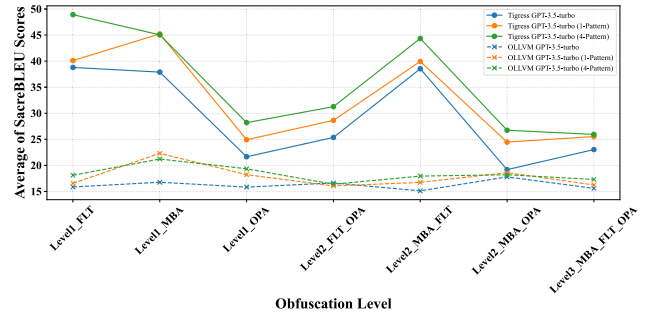


FIGURE 2. Comparison of SacreBLEU Scores for GPT-3.5-turbo across different prompt engineering strategies: no-pattern, 1-pattern, and 4-patterns. Results are evaluated for Tigress and OLLVM obfuscation levels.

and difficulty levels, restoring them to the original code more effectively.

Overall, the findings clearly demonstrate the important role that Prompt Patterns play in enhancing the model's deobfuscation performance. When all four patterns are employed—even for complex obfuscation methods like Level3_MBA_FLT_OPA—the model achieves notably high SacreBLEU Scores, maximizing deobfuscation performance. This provides strong evidence that LLM4DOBF, when used with effective Prompt Engineering, can deliver robust performance even in complex obfuscation environments.

B. SacreBLEU SCORE

SacreBLEU Score [56] is an evaluation metric designed to address the limitations of the BLEU (Bilingual Evaluation Understudy) Score [57], which is widely used in machine translation performance assessments. While BLEU measures the similarity between generated text and reference text based on n-gram overlap, its lack of standardized tokenization can lead to inconsistent experimental results. SacreBLEU mitigates this issue by applying fixed tokenization, ensuring greater consistency across different experiments and improving reproducibility by minimizing variations caused by data differences. Because SacreBLEU focuses on structural similarity between obfuscated and original code—evaluating factors such as variable names, function names, and code structure—it is particularly well-suited for measuring code restoration performance in deobfuscation tasks.

In this study, we use SacreBLEU to evaluate the performance of GPT-3.5, GPT-3.5*, GPT-4o, Claude-3.5, Gemini-1.5, and LLM4DOBF against the obfuscation tools Tigress and Obfuscator LLVM. As illustrated in Figure 4, LLM4DOBF achieves the highest SacreBLEU Scores across all obfuscation levels, demonstrating superior deobfuscation performance. Under Tigress, for instance, it outperforms GPT-3.5* by approximately 15% on average, even for complex obfuscation methods such as Level3_MBA_FLT_OPA, providing strong evidence of the effectiveness of the combined prompt engineering and fine-tuning approach.

It is important to clarify that the primary objective of this comparison is not merely to identify the top-performing model, but to validate the efficacy of our proposed

methodology. By intentionally applying fine-tuning only to the GPT-3.5-turbo based model, we aim to demonstrate that our approach can yield significant performance gains that surpass even more advanced, non-fine-tuned models.

By contrast, Gemini-1.5 and GPT-3.5-turbo exhibit relatively lower performance; notably, their SacreBLEU Scores decrease significantly in nested obfuscations that include Opaque Predicates. While Claude-3.5 performs comparatively well, its performance declines under complex obfuscation settings involving Opaque Predicates, with a particularly notable decrease in the Obfuscator LLVM environment. GPT-4o maintains a more uniform distribution of scores across obfuscation levels, but overall, it falls short of GPT-3.5* in performance.

Figure 3 presents a heatmap of SacreBLEU Scores comparing the Juliet C/C++ 1.3 dataset and the GitHub Obfuscation Benchmarks. The results indicate that the Obfuscation Benchmarks dataset yields higher SacreBLEU Scores than Juliet C/C++ 1.3, likely due to its more general code, which makes it easier for the LLM to reconstruct. In contrast, the Juliet C/C++ 1.3 dataset includes CWE scenarios that introduce structural complexity, posing greater challenges for restoring obfuscated code.

The strong performance of LLM4DOBF underscores the effectiveness of a strategy that integrates prompt engineering and fine-tuning. According to the SacreBLEU analyses, LLM4DOBF demonstrates more robust restoration capabilities than GPT-3.5* and other models across a range of obfuscation methods and complexity levels, particularly excelling at handling nested obfuscations. These findings confirm that LLM4DOBF is a highly effective solution for deobfuscation tasks and further validate SacreBLEU as a reliable performance evaluation metric.

C. OBFUSCATION QUALITY QUANTIFICATION FRAMEWORK

The Obfuscation Quality Quantification Framework [36] is an evaluation method designed to verify not only the structural restoration of obfuscated code by an LLM but also the accuracy of its performance. We employ this framework, using obfuscated code generated by Tigress and OLLVM, to quantitatively analyze the deobfuscation performance of LLMs. We use six primary metrics to evaluate the quality of code transformation. First, McCabe measures the overall complexity of the code, with lower values indicating improved readability and maintainability. Next, CFG Size represents the size of the Control Flow Graph, and smaller values suggest more streamlined control flow. Two further metrics, Program Length and Instruction Count, capture the total length of the code and the number of execution instructions, respectively, where lower values in both reflect higher levels of code optimization. The Code Optimization metric itself signifies the degree to which the code has been optimized; higher scores imply greater efficiency. Finally, Space Overhead gauges the memory footprint of the code,

and lower values correlate with more effective memory utilization.

According to the results for Tigress, presented in Table 3, the values for McCabe, CFG Size, Program Length, and Instruction Count are generally lower than those of the Original Code, suggesting that LLMs effectively remove dead code present in the original source. In addition to dead code elimination, the models successfully remove redundant operations, unnecessary control flows, and unoptimized loop structures. Notably, LLM4DOBF achieves the highest score in the Code Optimization metric, outperforming other models, including GPT-3.5*. In contrast, Gemini-1.5 and GPT-3.5-turbo exhibit relatively high Space Overhead values, indicating limitations in their ability to simplify obfuscated code complexity.

A similar trend is observed for OLLVM (Table 4). LLM4DOBF records a high Code Optimization score, along with reduced McCabe, CFG Size, and Instruction Count values compared to the Original Code. This outcome underscores LLM4DOBF's effectiveness in removing dead code, redundant operations, and inefficient code structures, while simultaneously simplifying obfuscated control flows. Meanwhile, Gemini-1.5 and GPT-4o exhibit increased Program Length and Space Overhead values relative to the Original Code, indicating constraints in their ability to generate optimized code.

Overall, in both Tigress and OLLVM environments, LLM4DOBF consistently eliminates dead code and other inefficient coding elements, while simplifying control flows, thereby producing more optimized code than the Original Code. Particularly notable is its high score in the Code Optimization metric, highlighting how LLM4DOBF substantially enhances code quality during deobfuscation. These findings confirm that LLM4DOBF serves as an effective solution for improving and optimizing complex obfuscated code, far surpassing the functionality of a mere deobfuscation tool.

D. LLMs COMPREHENSIVE PERFORMANCE EVALUATION

To comprehensively compare the performance of each model, we analyze the deobfuscation response rate and compilation success rate of various LLMs based on their No Response and Compile Error ratios. Table 5 presents these ratios when 980 obfuscated code samples generated by Tigress and OLLVM are supplied to the different LLMs.

LLM4DOBF achieves relatively low No Response ratios under Tigress (1.12%) and OLLVM (2.04%), significantly outperforming other models such as GPT-3.5-turbo-0125 (19.79%, 16.22%) and Claude-3.5-sonnet (13.06%, 34.38%). This result indicates that LLM4DOBF effectively analyzes obfuscated code patterns and maintains high accuracy throughout the deobfuscation process. By contrast, Gemini-1.5-flash exhibits a very low No Response ratio, but as shown in Figure 4, it registers comparatively lower SacreBLEU Scores, suggesting that, despite its high response rate, the quality of the generated code is suboptimal. While GPT-3.5* shows higher No Response ratios for Tigress (14.28%) and

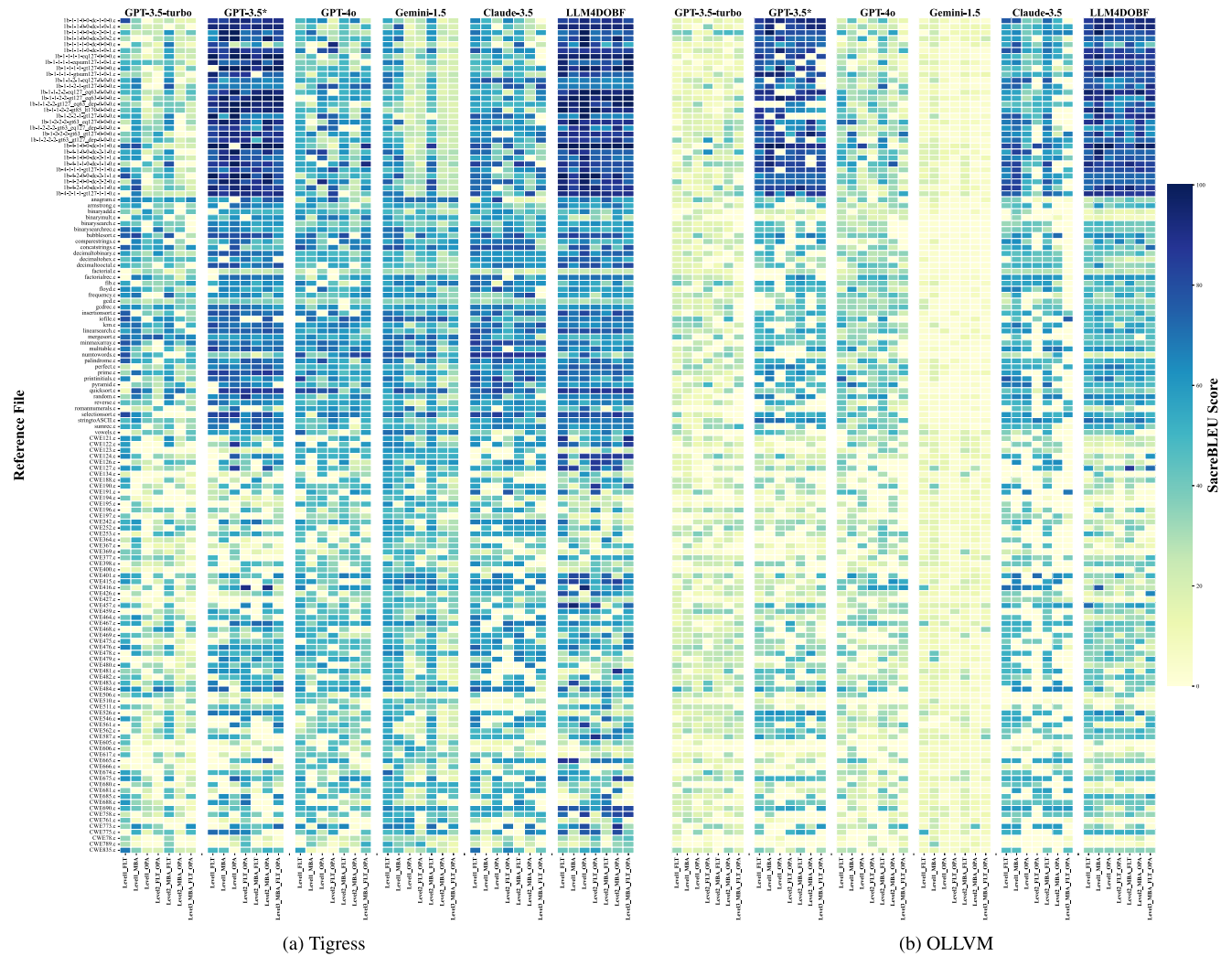


FIGURE 3. Heatmap comparison of SacreBLEU Scores for different LLMs across reference files and obfuscation levels using Tigress and OLLVM obfuscation tools. GPT-3.5* refers to the fine-tuned version of the GPT-3.5-turbo model.

OLLVM (23.57%) than LLM4DOBF, the finely tuned prompt engineering in LLM4DOBF results in significantly higher response rates, demonstrating the efficacy of structured prompt design in improving response performance.

Regarding Compile Error ratios, LLM4DOBF maintains relatively low values under Tigress (14.69%) and OLLVM (22.55%), ensuring stable performance compared to GPT-3.5-turbo-0125 (10.61%, 46.42%) and Claude-3.5-sonnet (37.55%, 10.71%). These findings suggest that LLM4DOBF effectively handles the complexity of obfuscated code and generates compilable outputs while preserving structural consistency. Analysis of compile errors reveals that most errors occur when the LLM incorrectly declares unknown header files, redundantly defines the main function, or mismanages type declarations—issues linked not only to code complexity but also to structural limitations in code reconstruction.

While GPT-4o achieves a relatively stable No Response ratio, it suffers from a high Compile Error ratio, indicating constraints in the quality and accuracy of its restored code.

The absence of dedicated prompt engineering and fine-tuning likely contributes to the model's limited understanding of structural consistency in obfuscated code.

LLM4DOBF consistently exhibits a high response rate and a low compile error ratio, and achieves top-tier performance in SacreBLEU-based evaluations for both deobfuscation accuracy and code quality. These results confirm that LLM4DOBF is not merely a model with a high response rate, but a comprehensive solution that delivers consistent, high-quality, and well-structured restored code. The combination of meticulously designed prompt engineering and fine-tuning emerges as a key factor behind its success, with comparisons against other models further highlighting the advantages of LLM4DOBF.

VI. DISCUSSION

In this section, we discuss the capacity of LLM4DOBF for continuous learning and highlight its key distinctions from existing deobfuscation methods. We explicitly address

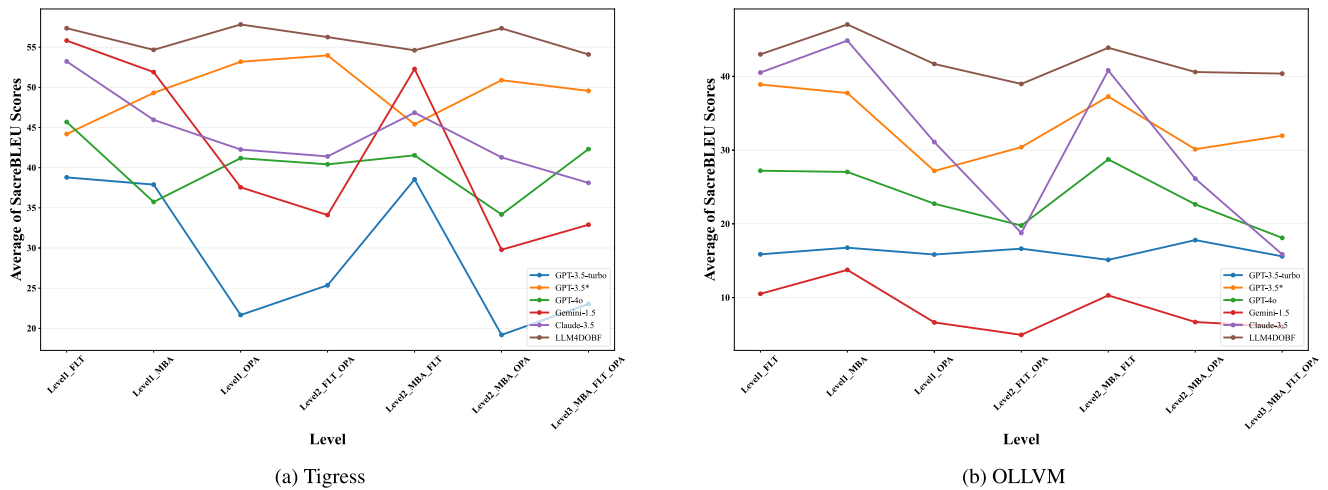


FIGURE 4. Comparison of SacreBLEU Scores for different LLMs across obfuscation levels using Tigress and OLLVM obfuscation tools. GPT-3.5* refers to the fine-tuned version of the GPT-3.5-turbo model.

the threats to validity and limitations of our study, particularly regarding the comparative analysis with traditional approaches. Through additional fine-tuning, LLM4DOBF is able to adapt to new obfuscation techniques while offering high temporal efficiency. In contrast, conventional methods face limitations in handling nested obfuscations, making direct comparison with LLM4DOBF impractical. We conclude by examining the tangible advantages and contributions of LLM4DOBF in deobfuscation tasks.

A. ADAPTIVE LEARNING AND UPDATING MECHANISMS

One of the key strengths of LLM4DOBF lies in its ability to support continuous learning and regular updates, enabling it to remain effective against newly emerging obfuscation methods. Large language models (LLMs) have an architecture that retains previously acquired knowledge from fine-tuning while incrementally improving performance through additional training data. This characteristic allows LLM4DOBF to continually adapt and maintain high performance, even as new obfuscation techniques emerge or existing ones become more complex.

When a new obfuscation method is introduced, the model's ability to respond is enhanced by further fine-tuning it with pairs of obfuscated code generated using the new method and their corresponding original code. LLM4DOBF is designed to learn new patterns effectively without compromising its previously acquired performance across established obfuscation techniques. Notably, this fine-tuning process requires relatively little data and training time, enabling LLM4DOBF to adapt more swiftly than the rate at which new obfuscation methods are typically developed. This not only enhances the practical applicability of the method but also ensures that it remains an efficient and scalable solution in rapidly evolving obfuscation environments.

Furthermore, by incorporating training data from diverse obfuscation tools, such as Tigress and OLLVM, during

fine-tuning, LLM4DOBF achieves a high degree of generality, preventing it from becoming overly specialized to a single tool or environment. Through regular updates and an efficient fine-tuning mechanism, LLM4DOBF consistently delivers stable and reliable results, even as obfuscation techniques evolve. Consequently, LLM4DOBF exemplifies a method capable of continuous learning and adaptation, providing an innovative and robust solution for both current and future obfuscation challenges.

B. GENERAL METHOD OF DEOBFUSCATION WITHOUT LLMs

Traditional deobfuscation techniques have been developed primarily to address individual obfuscation strategies in isolation. These include algebraic simplification rules for Mixed Boolean-Arithmetic (MBA) expressions, control-flow graph reconstruction for Control Flow Flattening, and symbolic or heuristic analysis for Opaque Predicates. While effective within their respective domains, these methods are typically not integrated and often rely on domain-specific assumptions that do not hold in nested or composite obfuscation scenarios.

To empirically examine the effectiveness of non-LLM-based deobfuscation, we attempted to reproduce prior work corresponding to each obfuscation category. However, we encountered several structural limitations. For Control Flow Flattening and Opaque Predicates deobfuscation, no publicly available tools exist that support automatic and reproducible deobfuscation under our current experimental conditions. These methods are often described only at a high level in the literature and lack sufficient implementation detail, making direct empirical comparison infeasible without re-implementing complex static or symbolic analyzers from scratch. As a result, only the MBA-related deobfuscation experiment could be conducted using the open-source tool GAMBA [58].

TABLE 3. Comparison of tigress obfuscation levels measured by the obfuscation quality quantification framework.

LLMs	Obfuscation Level	Metrics					
		MacCabe	CFG size	Program Length	Instr. Count	Code Opt.	Space Over.
Claude-3.5	Level1_FLT	5.52	10.98	42.59	68.47	67.84	2345.57
	Level1_MBA	6.35	12.95	46.22	77.90	71.10	2457.75
	Level1_OPA	5.87	12.23	44.10	75.13	71.74	2428.23
	Level2_FLT_OPA	5.39	10.91	47.42	74.18	67.60	2379.51
	Level2_MBA_FLT	5.43	10.63	42.09	69.76	68.43	2338.45
	Level2_MBA_OPA	5.50	10.90	41.57	69.97	72.85	2351.57
	Level3_FLT_MBA_OPA	5.74	10.57	40.73	67.48	71.29	2282.15
Gemini-1.5	Level1_FLT	6.40	12.91	60.24	81.16	64.41	2475.50
	Level1_MBA	6.75	14.06	61.75	87.33	64.06	2511.31
	Level1_OPA	7.19	14.85	89.46	123.43	66.92	2777.66
	Level2_FLT_OPA	6.47	13.37	83.09	113.36	64.76	2706.97
	Level2_MBA_FLT	5.96	12.25	60.71	84.43	65.15	2497.77
	Level2_MBA_OPA	6.34	13.11	97.31	139.59	63.85	2875.69
	Level3_FLT_MBA_OPA	6.16	12.38	82.60	114.86	66.20	2724.48
Gpt-3.5	Level1_FLT	6.12	11.84	48.03	69.48	65.36	2340.67
	Level1_MBA	5.45	10.53	49.19	76.85	62.02	2393.98
	Level1_OPA	5.97	11.36	61.08	95.04	63.54	2490.95
	Level2_FLT_OPA	5.87	11.12	57.53	84.74	64.94	2457.03
	Level2_MBA_FLT	5.08	9.64	44.91	70.67	60.94	2311.01
	Level2_MBA_OPA	5.55	10.35	63.28	104.44	63.51	2581.07
	Level3_FLT_MBA_OPA	5.61	10.57	56.29	88.86	63.79	2436.94
Gpt-3.5*	Level1_FLT	4.16	8.30	33.52	52.66	61.57	2123.52
	Level1_MBA	4.11	8.44	34.79	56.80	65.14	2177.43
	Level1_OPA	4.34	8.76	38.51	62.30	66.46	2223.83
	Level2_FLT_OPA	4.09	7.96	33.93	54.71	67.58	2132.40
	Level2_MBA_FLT	4.03	7.86	36.42	57.25	64.65	2170.74
	Level2_MBA_OPA	4.33	8.74	34.55	58.10	64.94	2155.40
	Level3_FLT_MBA_OPA	4.33	8.37	36.01	58.72	64.12	2172.53
Gpt-4o	Level1_FLT	7.29	13.84	60.38	85.69	64.39	2526.76
	Level1_MBA	8.06	16.13	64.86	92.28	62.49	2625.57
	Level1_OPA	6.07	12.52	49.12	77.14	68.86	2480.07
	Level2_FLT_OPA	5.95	12.21	49.36	76.58	69.65	2477.93
	Level2_MBA_FLT	6.65	12.42	54.64	77.29	63.09	2438.50
	Level2_MBA_OPA	6.39	12.93	51.90	81.26	71.37	2534.17
	Level3_FLT_MBA_OPA	6.13	12.34	47.29	74.69	68.81	2449.77
LLM4DOBF	Level1_FLT	4.30	8.61	37.24	56.62	64.53	2141.85
	Level1_MBA	4.06	8.00	36.10	56.31	65.00	2152.73
	Level1_OPA	4.84	9.35	49.25	61.60	65.73	2230.78
	Level2_FLT_OPA	4.52	8.65	40.10	58.11	68.10	2165.06
	Level2_MBA_FLT	3.78	7.23	31.97	51.62	66.86	2101.74
	Level2_MBA_OPA	4.37	8.44	39.60	58.59	65.21	2169.83
	Level3_FLT_MBA_OPA	4.20	8.34	36.50	55.68	63.41	2139.14
Obfuscation	Level1_FLT	24.16	39.00	281.06	131.15	68.75	3032.37
	Level1_MBA	7.96	15.28	173.77	174.11	47.75	2839.69
	Level1_OPA	7.96	15.28	340.88	351.06	56.87	4393.81
	Level2_FLT_OPA	24.16	39.00	402.52	363.04	60.51	4743.19
	Level2_MBA_FLT	24.16	39.00	281.06	204.43	55.00	3266.60
	Level2_MBA_OPA	7.96	15.28	332.09	401.39	51.28	4525.19
	Level3_FLT_MBA_OPA	24.16	39.00	401.49	433.62	54.67	4962.39
Original	Level0	6.66	13.44	72.35	80.75	66.42	2450.15

A key limitation of traditional tools like GAMBA is that they often require an analyst to manually identify and select the specific obfuscated expressions for analysis. In contrast, LLM4DOBF provides a more accessible approach that is easy to use even for analysts without this specialized domain knowledge. To illustrate the practical difference, we compared GAMBA's performance against LLM4DOBF on code

obfuscated with MBA using the Tigress obfuscator. In this experiment, LLM4DOBF achieved an average SacreBLEU score of 54.66, whereas GAMBA scored only 17.41. This significant discrepancy arises because obfuscation tools like Tigress do more than just apply MBA transformations; they also introduce holistic structural changes to the code, such as declaring new headers or altering internal logic.

TABLE 4. Comparison of OLLVM obfuscation levels measured by the obfuscation quality quantification framework.

LLMs	Obfuscation Level	Metrics					
		MacCabe	CFG size	Program Length	Instr. Count	Code Opt.	Space Over.
Claude-3.5	Level1_FLT	5.15	9.57	36.00	59.99	67.01	2246.18
	Level1_MBA	5.45	10.84	39.30	65.33	67.68	2316.11
	Level1_OPA	5.97	12.11	40.46	70.79	72.85	2363.73
	Level2_FLT_OPA	5.13	9.36	37.66	62.54	80.72	2300.16
	Level2_MBA_FLT	5.05	9.53	37.51	61.85	69.26	2268.28
	Level2_MBA_OPA	5.23	10.25	36.86	61.81	64.36	2256.52
	Level3_FLT_MBA_OPA	4.82	8.77	34.60	57.06	70.19	2257.92
Gemini-1.5	Level1_FLT	13.19	28.52	217.82	156.77	36.68	2890.30
	Level1_MBA	6.35	16.08	187.05	156.17	37.84	2807.04
	Level1_OPA	10.85	28.62	289.48	293.92	28.40	3340.18
	Level2_FLT_OPA	22.15	53.48	342.08	385.37	29.94	4120.42
	Level2_MBA_FLT	12.39	26.91	216.99	189.39	33.56	3066.95
	Level2_MBA_OPA	8.01	19.55	256.71	459.73	22.13	4157.80
	Level3_FLT_MBA_OPA	15.71	38.06	305.48	512.38	20.47	4759.91
Gpt-3.5	Level1_FLT	5.79	10.30	63.07	54.91	53.24	2110.86
	Level1_MBA	4.30	7.86	61.61	54.16	47.11	2058.32
	Level1_OPA	7.05	12.03	56.38	55.72	40.63	2058.93
	Level2_FLT_OPA	4.79	7.42	44.38	44.57	45.02	1988.17
	Level2_MBA_FLT	4.68	7.56	49.85	46.41	48.34	1985.45
	Level2_MBA_OPA	5.58	9.53	53.30	59.41	51.89	2133.48
	Level3_FLT_MBA_OPA	5.25	8.55	51.41	47.94	49.38	2048.71
Gpt-3.5*	Level1_FLT	4.16	8.55	30.30	52.74	66.69	2072.70
	Level1_MBA	4.12	8.80	34.79	56.80	65.14	2177.43
	Level1_OPA	4.25	8.76	38.51	62.30	66.46	2223.83
	Level2_FLT_OPA	4.09	7.96	33.93	54.71	67.58	2132.40
	Level2_MBA_FLT	4.03	7.86	36.42	57.25	64.65	2170.74
	Level2_MBA_OPA	4.33	8.74	34.55	58.10	64.94	2155.40
	Level3_FLT_MBA_OPA	4.33	8.37	36.01	58.72	64.12	2172.53
Gpt-4o	Level1_FLT	5.80	11.73	55.78	70.47	66.17	2394.03
	Level1_MBA	5.67	11.08	57.31	71.32	64.90	2406.85
	Level1_OPA	7.09	14.01	59.06	75.07	66.35	2430.13
	Level2_FLT_OPA	5.71	10.89	51.04	72.82	64.21	2369.32
	Level2_MBA_FLT	4.94	9.78	46.58	62.65	65.25	2290.01
	Level2_MBA_OPA	6.48	12.59	53.00	71.54	61.24	2393.14
	Level3_FLT_MBA_OPA	5.86	10.83	51.67	68.15	64.78	2341.71
LLM4DOBF	Level1_FLT	3.98	8.15	29.53	50.95	68.25	2081.85
	Level1_MBA	3.97	8.21	32.36	53.74	67.34	2117.31
	Level1_OPA	3.46	7.67	25.85	45.98	59.81	1997.97
	Level2_FLT_OPA	3.48	7.55	26.37	44.29	57.67	2013.79
	Level2_MBA_FLT	4.61	9.11	30.26	52.95	65.72	2095.78
	Level2_MBA_OPA	3.21	6.96	24.28	39.95	60.01	1950.38
	Level3_FLT_MBA_OPA	3.52	7.71	27.44	45.68	64.52	2022.06
Obfuscation	Level1_FLT	19.81	49.61	286.79	221.37	29.85	3405.34
	Level1_MBA	7.46	25.01	248.08	213.52	26.25	3153.31
	Level1_OPA	15.80	54.32	431.23	581.53	18.50	5109.14
	Level2_FLT_OPA	42.06	105.97	527.98	744.51	23.88	6632.28
	Level2_MBA_FLT	19.81	49.61	299.31	278.57	24.72	3748.20
	Level2_MBA_OPA	16.28	55.99	528.79	1690.66	7.84	11443.26
	Level3_FLT_MBA_OPA	40.60	102.00	589.90	1633.33	12.23	11736.91
Original	Level0	6.66	13.44	72.35	80.75	66.42	2450.15

GAMBA, as a deobfuscation methodology focused specifically on Mixed Boolean-Arithmetic, correctly handles the MBA expressions themselves but is unaware of the broader, structural modifications made to the rest of the file. Because the SacreBLEU score measures the overall similarity of the entire source code file against the original, GAMBA's output—while mathematically correct in its narrow focus—is rated poorly from a holistic code

deobfuscation perspective. In contrast, LLM4DOBF analyzes the entire code context, allowing it to not only reverse the MBA patterns but also to reconstruct a file that is structurally much closer to the original. This highlights the limitations of such specialized deobfuscation tools and underscores the advantage of a comprehensive, context-aware approach like LLM4DOBF for achieving true code deobfuscation.

TABLE 5. No response and compile error.

Model	No Response		Compile Error	
	Tigress	OLLVM	Tigress	OLLVM
GPT-3.5-turbo	19.79%	16.22%	10.61%	46.42%
GPT-4o	9.48%	19.59%	16.12%	27.44%
GPT-3.5*	14.28%	23.57%	10%	18.06%
Gemini-1.5	1.02%	0.3%	13.36%	52.55%
Claude-3.5	13.06%	34.38%	37.55%	10.71%
LLM4DOBF	1.12%	2.04%	14.69%	22.55%

VII. RELATED WORKS

This section examines existing deobfuscation methodologies and discusses their limitations. Traditional approaches rely heavily on manual intervention and heuristics by analysts to handle obfuscation patterns such as Mixed Boolean-Arithmetic (MBA) and complex control flows, significantly reducing their scope and efficiency when dealing with multilayered obfuscation. While these methods may be effective for specific types of obfuscation, they still face limitations in environments with multiple overlapping obfuscation methods.

A. LEVERAGING LLMs IN SECURITY FIELDS

Recent advances in large language models (LLMs) have underscored their growing importance in the security domain, harnessing contextual understanding and reasoning capabilities for various applications. One notable case is CHATAFL [27], which integrates LLMs into protocol fuzzing by extracting machine-readable grammar and generating state-transition messages. This approach surpasses the limitations of existing fuzzers (AFLNET, NSFUZZ), enabling an average 47.60% increase in explored state transitions, 5.81~6.74% greater code coverage, and the discovery of nine new vulnerabilities. Such results demonstrate how LLMs can complement traditional fuzzing techniques and detect a broader range of security issues more effectively.

Another example is LLM4Decompile [59], an innovative approach that uses LLMs as decompilers to transform binary code into human-readable source code. In experiments with various C-language binary datasets, a 6B-parameter model achieved an 87% recompilability rate and a 21% re-executability rate. Here, recompilability indicates syntactic comprehension, while re-executability reflects semantic preservation—showing that LLMs can successfully restore code beyond the reach of conventional decompilers. This study marks a new milestone in data-driven decompilation, confirming the potential of LLM-based decompiler development.

Lastly, DOBF enhances LLM performance by proposing a novel pre-training objective for code restoration. By integrating the task of restoring obfuscated code into pre-training, DOBF outperforms conventional Masked Language Modeling (MLM) with a 12.2% improvement in code translation accuracy and a 5.3% increase in code retrieval

performance. Designed to help LLMs more effectively learn the structure and semantics of obfuscated code, DOBF demonstrates strong results across multiple programming languages.

Collectively, these studies affirm the utility and effectiveness of LLMs for tackling complex security problems such as protocol fuzzing, binary decompilation, and code restoration. Although these models have shown promise in addressing various security challenges, no prior research has directly applied LLMs to source code deobfuscation. In this context, LLM4DOBF holds particular significance as the first approach to leverage LLMs for source code deobfuscation. By reconstructing the meaning and structure of obfuscated code and accommodating a range of obfuscation methods, LLM4DOBF presents a powerful solution in the deobfuscation landscape.

B. EXISTING DEOBFUSCATION METHODOLOGIES

Existing deobfuscation methods continue to rely heavily on the analyst's heuristics. Specifically, the process of identifying and defining obfuscated expressions or particular code segments still requires manual involvement, making it difficult to fully automate the deobfuscation process. This dependence on analysts is also a common limitation in studies of Mixed Boolean Arithmetic (MBA) deobfuscation. For instance, ProMBA [14]—evaluated for performance against MBASolver and Syntia—employs the same workflow as these earlier methods, requiring the analyst to locate obfuscated MBA expressions manually and then perform deobfuscation on each expression unit. As a result, when complicated obfuscation is layered across an entire codebase, the scope of deobfuscation becomes limited, and overall efficiency is significantly reduced.

Both MBASolver and Syntia simplify obfuscated expressions by exploiting SMT solvers and program synthesis, respectively. ProMBA expands on these ideas by combining program synthesis with term rewriting. However, none of these methods provide automated mechanisms for detecting or analyzing obfuscated expressions; instead, the burden still falls on the analyst to identify them first.

Studies on control-flow obfuscation at the source code level face similar constraints. Although SCORE suggests a tool to restore and improve the readability of obfuscated control flow, handling highly complex control flows still demands that the analyst identify and define the structure manually. While these methods may effectively handle certain types of obfuscation, they encounter inherent difficulties when multiple obfuscation methods are interwoven.

In sum, despite proposing effective approaches for specific obfuscation methods, current deobfuscation research commonly necessitates analyst intervention and struggles to handle nested obfuscations comprehensively.

VIII. CONCLUSION

In this study, we proposed LLM4DOBF, a novel method that leveraged large language models (LLMs) to effectively

restore obfuscated code. By integrating fine-tuning and prompt engineering, LLM4DOBF maximized the deobfuscation performance of LLMs, minimized the need for manual analyst intervention, and achieved high adaptability and scalability across diverse obfuscation tools and techniques. This approach overcame the limitations of automation and generalization observed in existing methods, providing stable and efficient performance even in complex obfuscation environments.

Our experimental results demonstrated the syntactic and semantic accuracy of LLM4DOBF using SacreBLEU scores, while the Obfuscation Quality Quantification Framework quantitatively verified its structural optimization and restoration quality. LLM4DOBF recorded an average No Response rate of just 1.12%, indicating high reliability in producing valid outputs and maintaining robust performance under various mixed obfuscation scenarios. These findings confirmed LLM4DOBF's superior performance and stability, underscoring its effectiveness in overcoming the constraints of conventional deobfuscation methods.

Moreover, we validated LLM4DOBF's broad adaptability by evaluating it on multiple LLMs and across different obfuscation tools and techniques. Future work will explore new and increasingly complex obfuscation methods, further enhancing LLM4DOBF's practicality and generalizability.

REFERENCES

- [1] C. Collberg, C. Thomborson, and D. Low, "A taxonomy of obfuscating transformations," Dept. Comput. Sci., University of Auckland, Auckland, New Zealand, Tech. Rep. 148, 1997.
- [2] C. Collberg and C. Thomborson, "Software watermarking: Models and dynamic embeddings," in *Proc. 26th ACM SIGPLAN-SIGACT Symp. Princ. Program. Lang.*, Jan. 1999, pp. 311–324.
- [3] M. Hammad, J. Garcia, and S. Malek, "A large-scale empirical study on the effects of code obfuscations on Android apps and anti-malware products," in *Proc. IEEE/ACM 40th Int. Conf. Softw. Eng. (ICSE)*, May 2018, pp. 421–431.
- [4] C. S. Collberg and C. Thomborson, "Watermarking, tamper-proofing, and obfuscation—tools for software protection," *IEEE Trans. Softw. Eng.*, vol. 28, no. 8, pp. 735–746, Aug. 2002.
- [5] S. Li, C. Jia, P. Qiu, Q. Chen, J. Ming, and D. Gao, "Chosen-instruction attack against commercial code virtualization obfuscators," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2022, pp. 1–17.
- [6] P. O'Kane, S. Sezer, and K. McLaughlin, "Obfuscation: The hidden malware," *IEEE Secur. Privacy*, vol. 9, no. 5, pp. 41–47, Sep. 2011.
- [7] R. Kumar and A. R. E. Vaishakh, "Detection of obfuscation in Java malware," *Procedia Comput. Sci.*, vol. 78, pp. 521–529, Feb. 2016.
- [8] O. Or-Meir, N. Nissim, Y. Elovici, and L. Rokach, "Dynamic malware analysis in the modern Era—A state of the art survey," *ACM Comput. Surveys*, vol. 52, no. 5, pp. 1–48, Sep. 2020.
- [9] W. F. Elersy, A. Feizollah, and N. B. Anuar, "The rise of obfuscated Android malware and impacts on detection methods," *PeerJ Comput. Sci.*, vol. 8, p. e907, Mar. 2022.
- [10] T. Blazytko, M. Contag, C. Aschermann, and T. Holz, "Syntia: Synthesizing the semantics of obfuscated code," in *Proc. 26th USENIX Secur. Symp. (USENIX Security)*, 2017, pp. 643–659.
- [11] J. Salwan, S. Bardin, and M.-L. Potet, "Symbolic deobfuscation: From virtualized code back to the original," in *Proc. Int. Conf. Detection Intrusions Malware*. Cham, Switzerland: Springer, 2018, pp. 372–392.
- [12] B. Reichenwallner and P. Meerwald-Stadler, "Efficient deobfuscation of linear mixed Boolean-arithmetic expressions," in *Proc. ACM Workshop Res. offensive defensive Techn. context Man At End (MATE) attacks*, Nov. 2022, pp. 19–28.
- [13] J. H. Suk, Y. B. Lee, and D. H. Lee, "SCORE: Source code optimization & reconstruction," *IEEE Access*, vol. 8, pp. 129478–129496, 2020.
- [14] J. Lee and W. Lee, "Simplifying mixed Boolean-arithmetic obfuscation by program synthesis and term rewriting," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2023, pp. 2351–2365.
- [15] G. Menguy, S. Bardin, R. Bonichon, and C. D. S. Lima, "Search-based local black-box deobfuscation: Understand, improve and mitigate," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 2513–2525.
- [16] S. Schrittwieser and S. Katzenbeisser, "Code obfuscation against static and dynamic reverse engineering," in *Proc. Inf. Hiding: 13th Int. Conf.*, Prague, Czech Republic. Berlin, Germany: Springer, 2011, pp. 270–284.
- [17] B. Yadegari, "Automatic deobfuscation and reverse engineering of obfuscated code," Ph.D. dissertation, Dept. Comput. Sci., The University of Arizona, Tucson, AZ, USA, 2016.
- [18] X. Zhang, F. Breiting, E. Luechinger, and S. O'Shaughnessy, "Android application forensics: A survey of obfuscation, obfuscation detection and deobfuscation techniques and their impact on investigations," *Forensic Sci. International: Digit. Invest.*, vol. 39, Dec. 2021, Art. no. 301285.
- [19] Y. Zhou, A. Main, Y. Gu, and H. J. Johnson, "Information hiding in software with mixed Boolean-arithmetic transforms," in *Proc. Int. Workshop Inf. Secur. Appl.* Berlin, Germany: Springer, 2007, pp. 61–75.
- [20] S. Hosseinzadeh, S. Rauti, S. Laurén, J.-M. Mäkelä, J. Holvitie, S. Hyrynsalmi, and V. Leppänen, "Diversification and obfuscation techniques for software security: A systematic literature review," *Inf. Softw. Technol.*, vol. 104, pp. 72–93, Dec. 2018.
- [21] B. Yadegari, B. Johannesmeyer, B. Whitely, and S. Debray, "A generic approach to automatic deobfuscation of executable code," in *Proc. IEEE Symp. Secur. Privacy*, May 2015, pp. 674–691.
- [22] A. Afianian, S. Niksefat, B. Sadeghiyan, and D. Baptiste, "Malware dynamic analysis evasion techniques: A survey," *ACM Comput. Surveys*, vol. 52, no. 6, pp. 1–28, Nov. 2020.
- [23] P. Kochberger, S. Schrittwieser, S. Schweighofer, P. Kieseberg, and E. Weippl, "SoK: Automatic deobfuscation of virtualization-protected applications," in *Proc. 16th Int. Conf. Availability, Rel. Secur.*, Aug. 2021, pp. 1–15.
- [24] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, "Large language models for software engineering: A systematic literature review," *ACM Trans. Softw. Eng. Methodology*, vol. 33, no. 8, pp. 1–79, Nov. 2024.
- [25] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, and J. M. Zhang, "Large language models for software engineering: Survey and open problems," in *Proc. IEEE/ACM Int. Conf. Softw. Engineering: Future Softw. Eng. (ICSE-FOSE)*, May 2023, pp. 31–53.
- [26] Y. Deng, C. S. Xia, H. Peng, C. Yang, and L. Zhang, "Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models," in *Proc. 32nd ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, Jul. 2023, pp. 423–435.
- [27] R. Meng, M. Mirchev, M. Böhme, and A. Roychoudhury, "Large language model guided protocol fuzzing," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2024, pp. 1–17.
- [28] C. Patsakis, F. Casino, and N. Lykousas, "Assessing LLMs in malicious code deobfuscation of real-world malware campaigns," *Expert Syst. Appl.*, vol. 256, Dec. 2024, Art. no. 124912.
- [29] J. Shin, C. Tang, T. Mohati, M. Nayeibi, S. Wang, and H. Hemmati, "Prompt engineering or fine-tuning: An empirical assessment of LLMs for code," 2023, *arXiv:2310.10508*.
- [30] C. Pornprasit and C. Tantithamthavorn, "Fine-tuning and prompt engineering for large language models-based code review automation," *Inf. Softw. Technol.*, vol. 175, Nov. 2024, Art. no. 107523.
- [31] T. T. Kim, M. Makutonin, R. Sirous, and R. Javan, "Optimizing large language models in radiology and mitigating pitfalls: Prompt engineering and fine-tuning," *RadioGraphics*, vol. 45, no. 4, Apr. 2025, Art. no. 240073.
- [32] B. Choi, H. Jin, D. H. Lee, and W. Choi, "Chatdeob: An effective deobfuscation method based on large language model," in *Proc. Int. Conf. Inf. Secur. Appl.* Cham, Switzerland: Springer, 2024, pp. 151–163.
- [33] H. Xu, Y. Zhou, J. Ming, and M. Lyu, "Layered obfuscation: A taxonomy of software obfuscation techniques for layered security," *Cybersecurity*, vol. 3, no. 1, pp. 1–18, Dec. 2020.
- [34] T. Raitsis, Y. Elgazari, G. E. Toibin, Y. Lurie, S. Mark, and O. Margalit, "Code obfuscation: A comprehensive approach to detection, classification, and ethical challenges," *Algorithms*, vol. 18, no. 2, p. 54, Jan. 2025.
- [35] S. A. Ebad, A. A. Darem, and J. H. Abawajy, "Measuring software obfuscation quality—A systematic literature review," *IEEE Access*, vol. 9, pp. 99024–99038, 2021.

- [36] H. Jin, J. Lee, S. Yang, K. Kim, and D. H. Lee, "A framework to quantify the quality of source code obfuscation," *Appl. Sci.*, vol. 14, no. 12, p. 5056, Jun. 2024.
- [37] C. K. Behera and D. L. Bhaskari, "Different obfuscation techniques for code protection," *Procedia Comput. Sci.*, vol. 70, pp. 757–763, Oct. 2015.
- [38] S. K. Udupa, S. K. Debray, and M. Madou, "Deobfuscation: Reverse engineering obfuscated code," in *Proc. 12th Work. Conf. Reverse Eng. (WCRE05)*, 2005, pp. 45–54.
- [39] A. Majumdar and C. Thomborson, "Manufacturing opaque predicates in distributed systems for code obfuscation," in *Proc. 29th Australas. Comput. Sci. Conf.*, 2006, pp. 187–196.
- [40] OpenAI. (2024). *GPT-4o System Card*. [Online]. Available: <https://openai.com/index/gpt-4o-system-card/>
- [41] Google. (2024). *Our Next-generation Model: Gemini 1.5*. [Online]. Available: <https://blog.google/technology/ai/google-gemini-next-generation-model-february-2024/>
- [42] Anthropic, "Claude 3.5 sonnet," 2024. <https://www.anthropic.com/claude/sonnet>
- [43] Z. Ding, A. Srinivasan, S. Macneil, and J. Chan, "Fluid transformers and creative analogies: Exploring large language models' capacity for augmenting cross-domain analogical creativity," in *Proc. Creativity Cognition*, Jun. 2023, pp. 489–505.
- [44] B. Kolosnjaji, A. Zarras, G. D. Webster, and C. Eckert, "Deep learning for classification of malware system call sequences," in *Proc. AI : Adv. Artif. Intelligence: 29th Australas. Joint Conf.*, Hobart, TAS, Australia. Cham, Switzerland: Springer, 2016, pp. 137–149.
- [45] H. Xu, S. Wang, N. Li, K. Wang, Y. Zhao, K. Chen, T. Yu, Y. Liu, and H. Wang, "Large language models for cyber security: A systematic literature review," 2024, *arXiv:2405.04760*.
- [46] H. Jelodar, S. Bai, P. Hamed, H. Mohammadian, R. Razavi-Far, and A. Ghorbani, "Large language model (LLM) for software security: Code analysis, malware analysis, reverse engineering," 2025, *arXiv:2504.07137*.
- [47] J. Saxe and K. Berlin, "Deep neural network based malware detection using two dimensional binary program features," in *Proc. 10th Int. Conf. Malicious Unwanted Softw. (MALWARE)*, Oct. 2015, pp. 11–20.
- [48] E. Basic and A. Giarretta, "From vulnerabilities to remediation: A systematic literature review of LLMs in code security," 2024, *arXiv:2412.15004*.
- [49] M. A. Ferrag, F. Alwahedi, A. Battah, B. Cherif, A. Mechri, and N. Tihanyi, "Generative ai and large language models for cyber security: All insights you need," *SSRN*, 2024.
- [50] Q. Ma, W. Peng, C. Yang, H. Shen, K. Koedinger, and T. Wu, "What should we engineer in prompts? Training humans in requirement-driven LLM use," 2024, *arXiv:2409.08775*.
- [51] I. Joshi, S. Shahid, S. M. Venneti, M. Vasu, Y. Zheng, Y. Li, B. Krishnamurthy, and G. Y.-Y. Chan, "CoPrompter: User-centric evaluation of LLM instruction alignment for improved prompt engineering," in *Proc. 30th Int. Conf. Intell. User Interfaces*, Mar. 2025, pp. 341–365.
- [52] OpenAI. (2023). *Fine-Tuning*. [Online]. Available: <https://platform.openai.com/docs/guides/fine-tuning>
- [53] J. White, Q. Fu, S. Hays, M. Sandborn, C. Olea, H. Gilbert, A. Elnashar, J. Spencer-Smith, and D. C. Schmidt, "A prompt pattern catalog to enhance prompt engineering with ChatGPT," 2023, *arXiv:2302.11382*.
- [54] National Institute of Standards and Technology. (2017). *Software Assurance Reference Dataset Test Suites*. [Online]. Available: <https://samate.nist.gov/SARD/test-suites/112>
- [55] Technical University of Munich. (2016). *Obfuscation Benchmarks*. [Online]. Available: <https://github.com/tum-i4/obfuscation-benchmarks>
- [56] M. Post, "A call for clarity in reporting BLEU scores," 2018, *arXiv:1804.08771*.
- [57] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "Bleu: A method for automatic evaluation of machine translation," in *Proc. 40th Annu. Meeting Assoc. Comput. Linguistics*, 2002, pp. 311–318.
- [58] B. Reichenwallner and P. Meerwald-Stadler, "Simplification of general mixed Boolean-arithmetic expressions: GAMBA," in *Proc. IEEE Eur. Symp. Secur. Privacy Workshops (EuroSPW)*, Jul. 2023, pp. 427–438.
- [59] H. Tan, Q. Luo, J. Li, and Y. Zhang, "LLM4Decompile: Decompiling binary code with large language models," 2024, *arXiv:2403.05286*.



BYUNGGEON CHOI received the B.S. degree in information security from Sejong University, Seoul, South Korea, in 2023. He is currently pursuing the M.S. degree in convergence security with the Graduate School of Information Security, Korea University, Seoul. His research interests include source code obfuscation and large language models.



HONGJOO JIN received the B.S. degree in computer engineering from Hongik University, Seoul, South Korea, in 2017, the M.S. degree in information security from Korea University, Seoul, in 2019, and the Ph.D. degree in information security from the Graduate School of Information Security, Korea University, in 2025. Since 2025, he has been with Samsung Electronics. His research interests include virtual memory protection, obfuscation, and large language model.



DONG HOON LEE received the B.S. degree from the Department of Economics, Korea University, Seoul, South Korea, in 1985, and the M.S. and Ph.D. degrees in computer science from The University of Oklahoma, USA, in 1988 and 1992, respectively. Since 1993, he has been with the Faculty of Computer Science and Information Security, Korea University. He is currently a Professor and the Director of the Graduate School of Information Security, Korea University. His research interests include cryptographic protocol, applied cryptography, functional encryption, software protection, mobile security, vehicle security, and ubiquitous sensor network (USN) security.



WONSUK CHOI received the B.S. degree in mathematics from the University of Seoul, Seoul, South Korea, in 2008, and the M.S. and Ph.D. degrees in information security from the School of Cybersecurity, Korea University, Seoul, in 2013 and 2018, respectively. He was a Post-doctoral Researcher with the Institute of Cyber Security and Privacy, Korea University. In 2020, he joined Hansung University and moved to the School of Cybersecurity, Korea University, in 2023. His research interests include automotive security, drone security, and robot security, which are all aspects of mobility security. He served as a Board Member for Korea Institute of Information Security and Cryptology (KIISC).

...